

Imperial College London
Department of Earth Science and Engineering
MSc in Applied Computational Science and Engineering

Independent Research Project
Final Report

Solving the Wave Equation by Optimising a Discrete Loss

by

Luke Dinsdale

repository: <https://github.com/ease-ada-lovelace-2025/irp-ld2022>

Supervisors:

Dr Lluís Guasch Batalla

Dr Ben Moseley

Dr Carlos Cueto Mondejar

August 2026

Abstract

The numerical modelling of waves has broad applications across science and engineering. Recently, the Optimising a Discrete Loss (ODIL) framework has been proposed as an efficient method for producing accurate approximations of the solution of Partial Differential Equations (PDE) by means of minimising a discrete functional that expresses the PDE and its conditions in residual form. ODIL does not require a time stepping approach, unlike classical numerical methods, which may offer accuracy and efficiency benefits when solving the wave equation. The present work presents a methodology for solving the acoustic wave equation in two dimensions using the ODIL framework and establishes a set of guidelines for using it.

1 Introduction

The modern world depends strongly on our understanding of wave propagation. With applications spanning medical imaging, resource prospecting, and telecommunications, the implications of wave propagation are countless and have been central to the development of civilisation. Supporting this development is our ability to solve the wave equation numerically, that is, to approximate it by a discrete form so that it may be represented on a computer.

While an analytical solution to the wave equation exists under certain assumptions, practical problems require the flexibility to represent complex geometries, boundary conditions, and forcing terms, so the wave equation is instead solved numerically by approximating it with a discrete form. The finite difference [29, 6] and finite element [33] methods are the standard approaches, producing systems of equations that are solved either iteratively [20] or directly [16]. Both incur a computational cost that can be substantial, owing to the often ill-conditioned and sparse nature of the discretised systems [27, 23, 15].

Beyond cost, finite difference schemes for the wave equation carry their own characteristic errors. Discretisation introduces numerical dispersion and dissipation [26], causing waves to propagate at an incorrect speed, and because the wave equation is solved via time stepping, these errors accumulate both with propagation distance and over long time horizons [2]. Suppressing them typically demands finer discretisation [16], which drives the computational cost up further. High order methods [5], dispersion reduction schemes [11, 3], and spectral methods [4] each address one side of this trade off, but the underlying tension between accuracy and cost persists.

Rather than refining time stepping methodologies, an alternative approach is to reformulate the problem as an optimisation problem. Optimising a Discrete Loss (ODIL) [22] can approximate the solution to a PDE by means of minimising a functional $\mathcal{L}$ that encapsulates the discretised PDE, and boundary and initial conditions. In the functional, the PDE is expressed in a residual form in terms of the field of interest, which is optimised directly. Thus, if a minimiser drives the loss to zero, it is the solution field which satisfies the PDE and boundary conditions exactly.

The fundamental difference between ODIL and time stepping is that the objective of minimising a residual means that we do not necessarily enforce the wave equation exactly at each time level. Instead, we find the pressure field which most closely satisfies the wave equation according to the functional. As a result, when applied to the wave equation, ODIL may have different stability properties to explicit time stepping methods. The solutions obtained by such methods destabilise entirely when the Courant-Friedrichs-Lewy (CFL) condition is violated, but since ODIL does not require the PDE to be satisfied exactly, its solution to the wave equation may behave differently in regimes where the CFL condition would suggest the solution should destabilise.

Whether the ODIL formulation provides any practical advantage or disadvantage when solving the wave equation is yet to be determined. Karnakov et al. [22], demonstrated ODIL for PDE problems including solving the homogeneous wave equation in 1 dimension, among others. The

framework’s accuracy and computational cost properties was compared against that of Physics Informed Neural Networks (PINN). However, a systematic comparison against traditional time stepping methodologies has not been established. In addition, how ODIL treats known sources of errors, such as dispersion and dissipation in the wave equation, is yet to be studied in detail. The present work aims to compare ODIL against time stepping for solving the wave equation in these areas by addressing three defining Research Questions (RQ):

- RQ1: How does the accuracy of ODIL compare with that of time stepping methods?
- RQ2: How does the computational cost of ODIL compare to that of time stepping methods?
- RQ3: What are the stability properties of ODIL when applied to wave modelling, and how does it treat known sources of error?

The present work establishes a comparison between ODIL and finite difference time stepping on the basis of accuracy, performance, and wave-specific errors. A hardware parallelised, preconditioned methodology is implemented for solving ODIL problems involving the 2D wave equation. We compare our implementation against a modern finite difference engine, Devito [25], a highly optimised library designed primarily for time stepping wave modelling. On the basis of this study, we propose a practical set of guidelines for using the ODIL framework to solve the forward wave equation in two dimensions. In support of our objectives and as a tool for further research, we present a modular, extensible Python library for solving the 2D wave equation using the ODIL framework.

2 Methods

2.1 The Acoustic Wave Equation

Mechanical wave propagation through a medium can be modelled using the linear acoustic wave equation. Assuming a lossless, isotropic medium, the pressure field $u = u(t, x, y)$ satisfies

$$\frac{\partial^2 u}{\partial t^2} - c^2 \nabla^2 u = s, \quad (1)$$

where $c = c(x, y)$ is the spatially varying wave speed, ∇^2 denotes the two-dimensional Laplacian operator,

$$\nabla^2 = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}, \quad (2)$$

with time t and the spatial coordinates (x, y) . $s = s(t)$ is the acoustic source modelled as a time shifted Ricker wavelet [28],

$$s(t) = (1 - 2\alpha) e^{-\alpha}, \quad (3)$$

$$\alpha = \pi^2 f_0^2 (t - t_0)^2, \quad (4)$$

where f_0 is the peak frequency and t_0 is the time shift that centres the pulse.

2.1.1 Initial and Boundary Conditions

The pressure field is assumed to be initially at rest, corresponding to zero initial pressure and zero initial particle velocity prior to excitation by the acoustic source. Correspondingly, we subject the wave equation to the initial conditions

$$u(0, x, y) = 0 \quad (5)$$

$$\frac{\partial}{\partial t} u(0, x, y) = 0 \quad (6)$$

At the boundaries, we impose second order Higdon absorbing boundary conditions [19]. They provide improved absorption of outgoing waves over a wider range of incidence angles than first order absorbing conditions while remaining significantly less computationally expensive than perfectly matched layers. They therefore provide a suitable balance between accuracy and computational cost for the present work.

In two dimensions, the second order Higdon absorbing boundary conditions read

$$\left(\frac{\partial}{\partial t} + c_1 \frac{\partial}{\partial n} \right) \left(\frac{\partial}{\partial t} + c_2 \frac{\partial}{\partial n} \right) u = 0, \quad (7)$$

where the normal derivative

$$\frac{\partial u}{\partial n} = \nabla u \cdot \mathbf{n}, \quad (8)$$

is the spatial derivative of the pressure field in the direction perpendicular to the boundary with $\mathbf{n}$ as the outward unit normal. The parameters c_1 and c_2 represent the propagation velocities of outgoing waves that the boundary condition is designed to absorb. For a homogeneous medium, these are commonly chosen to be

$$c_n = c \cos(\theta_n), \quad (9)$$

where θ_n is the angle of incidence relative to the boundary normal.

In Appendix A, we show that Equation (7) can be written as

$$\frac{\partial^2 u}{\partial t^2} + (c_1 + c_2) \frac{\partial^2 u}{\partial t \partial n} + c_1 c_2 \frac{\partial^2 u}{\partial n^2} = 0. \quad (10)$$

where mixed partial derivatives have been combined using the commutativity of partial differentiation. We use this form because it is much easier to discretise, since each derivative can be evaluated independently according to predefined finite difference operators which we discuss in 2.3.1.

2.2 Optimising a Discrete Loss

Unlike conventional finite difference methods, which solve the wave equation by explicitly marching forward in time, ODIL reformulates the governing PDE as an optimisation problem over the complete discrete space time domain. The discrete field itself is treated as the optimisation variable, and the solution is obtained by minimising a loss function constructed from the residual of the discretised governing equations.

In Section 2.3, we show that the wave equation can be written as a discrete linear system

$$A\mathbf{u} = \mathbf{s}, \quad (11)$$

where A encodes the action of discrete approximations of the derivative operators and the aforementioned initial and boundary conditions, $\mathbf{u}$ (the optimisation variable) is a discrete representation of our pressure field, and $\mathbf{s}$ is our modelled source. Accordingly, the discrete residual can be written

$$r(\mathbf{u}) = A\mathbf{u} - \mathbf{s}. \quad (12)$$

In the ODIL framework, the optimisation variable comprises the entire discrete space-time pressure field rather than the solution at a single time level, as is the case in conventional time stepping methods. Hence, we choose to write the wave equation in the form of an all-at-once system as above.

Under this formulation, the discrete loss function $\mathcal{L}$ is

$$\mathcal{L}(\mathbf{u}) = \frac{1}{2} \|\mathbf{r}(\mathbf{u})\|^2, \quad (13)$$

so the residual vanishes when the discrete wave equation is satisfied. The forward problem we seek to solve is therefore the unconstrained optimisation problem

$$\min_{\mathbf{u}} \mathcal{L}(\mathbf{u}). \quad (14)$$

Then it follows that the minimiser is a discrete pressure field $\mathbf{u}$ for which the residual is minimised and, in the ideal case, vanishes entirely, implying satisfaction of the discretised wave equation.

2.3 Finite Difference Discretisation

The continuous domain is discretised into a finite set of spatial and temporal grid points. The resulting discrete approximation replaces the continuous differential operators with finite difference operators, allowing the wave equation to be expressed as a finite dimensional system suitable for numerical solution.

The computational domain is discretised using a uniform Cartesian grid with N_t , N_x , N_y grid nodes in the temporal and spatial directions, respectively. The spatial coordinates are then defined as

$$x_i := i\Delta x, \quad y_j := j\Delta y \quad (15)$$

and temporal coordinates

$$t_n := n\Delta t, \quad (16)$$

with $n = 0, 1, \dots, N_t - 1$ (and analogously for x and y) and where Δx , Δy , and Δt denote the spatial and temporal grid spacings, respectively.

Under this discretisation, our discrete pressure field $u_{i,j}^n$ becomes an approximation of the continuous pressure field u :

$$u_{i,j}^n \approx u(t_n, x_i, y_j), \quad (17)$$

The spatial and temporal second derivatives in Equation (1) are discretised with standard centred finite differences. In practice we employ higher-ordered schemes (4th, 6th, 8th) detailed in Appendix B, but for the purpose of this discussion we will make reference to the 2nd order central differences,

$$\frac{\partial^2 u}{\partial t^2} \approx \frac{u_{i,j}^{n+1} - 2u_{i,j}^n + u_{i,j}^{n-1}}{\Delta t^2} \quad (18)$$

$$\frac{\partial^2 u}{\partial x^2} \approx \frac{u_{i+1,j}^n - 2u_{i,j}^n + u_{i-1,j}^n}{\Delta x^2} \quad (19)$$

$$\frac{\partial^2 u}{\partial y^2} \approx \frac{u_{i,j+1}^n - 2u_{i,j}^n + u_{i,j-1}^n}{\Delta y^2} \quad (20)$$

2.3.1 Matrix Representation of the Discrete Operators

The finite difference discretisation transforms the continuous wavefield into a set of unknown values defined on the discrete space-time grid. These values are collected into a vector representation,

$$\mathbf{u} := \left[(u^0)^T, (u^1)^T, \dots, (u^{N_t-1})^T \right]^T. \quad (21)$$

Here, the spatial indices are flattened into a single spatial degree of freedom index where $u^n \in \mathbb{R}^{N_x N_y}$ is the pressure field across space at the n -th time level. Under this representation, the differential operators are naturally represented as sparse matrices acting on the global vector.

We define the temporal second derivative operator $D_{tt} \in \mathbb{R}^{N_t \times N_t}$ and the second spatial derivative operators $D_{xx} \in \mathbb{R}^{N_x \times N_x}$ and $D_{yy} \in \mathbb{R}^{N_y \times N_y}$. These matrices represent the action of the finite difference approximations of second derivatives on $\mathbf{u}$. In the absence of boundary modifications, the finite difference derivative matrices possess a Toeplitz structure, illustrated in Figure 1.

The global matrix operator A is formed by applying Kronecker products to form a global matrix operator $A \in \mathbb{R}^{N_t N_x N_y \times N_t N_x N_y}$ that represents the action of the second temporal derivative and Laplacian across the full space-time vector,

$$A := D_{tt} \otimes I_{xy} - I_t \otimes (C^2 L_{xy}), \quad (22)$$

where $I_t \in \mathbb{R}^{N_t \times N_t}$ and $I_{xy} \in \mathbb{R}^{N_x \times N_y}$ are the identity matrices and $C \in \mathbb{R}^{N_x N_y \times N_x N_y}$ is a diagonal matrix containing the spatially varying wave speed values on the computational grid. $L_{xy} \in \mathbb{R}^{N_x N_y \times N_x N_y}$ is the Laplacian operator whose construction is detailed in Appendix C. Note that in practice we never form this global operator. Instead, we store its constituent operator matrices and apply the action of A through a series of matrix-vector products. Figure 1 demonstrates the structure of the global operator A and the constituent temporal and spatial operators.

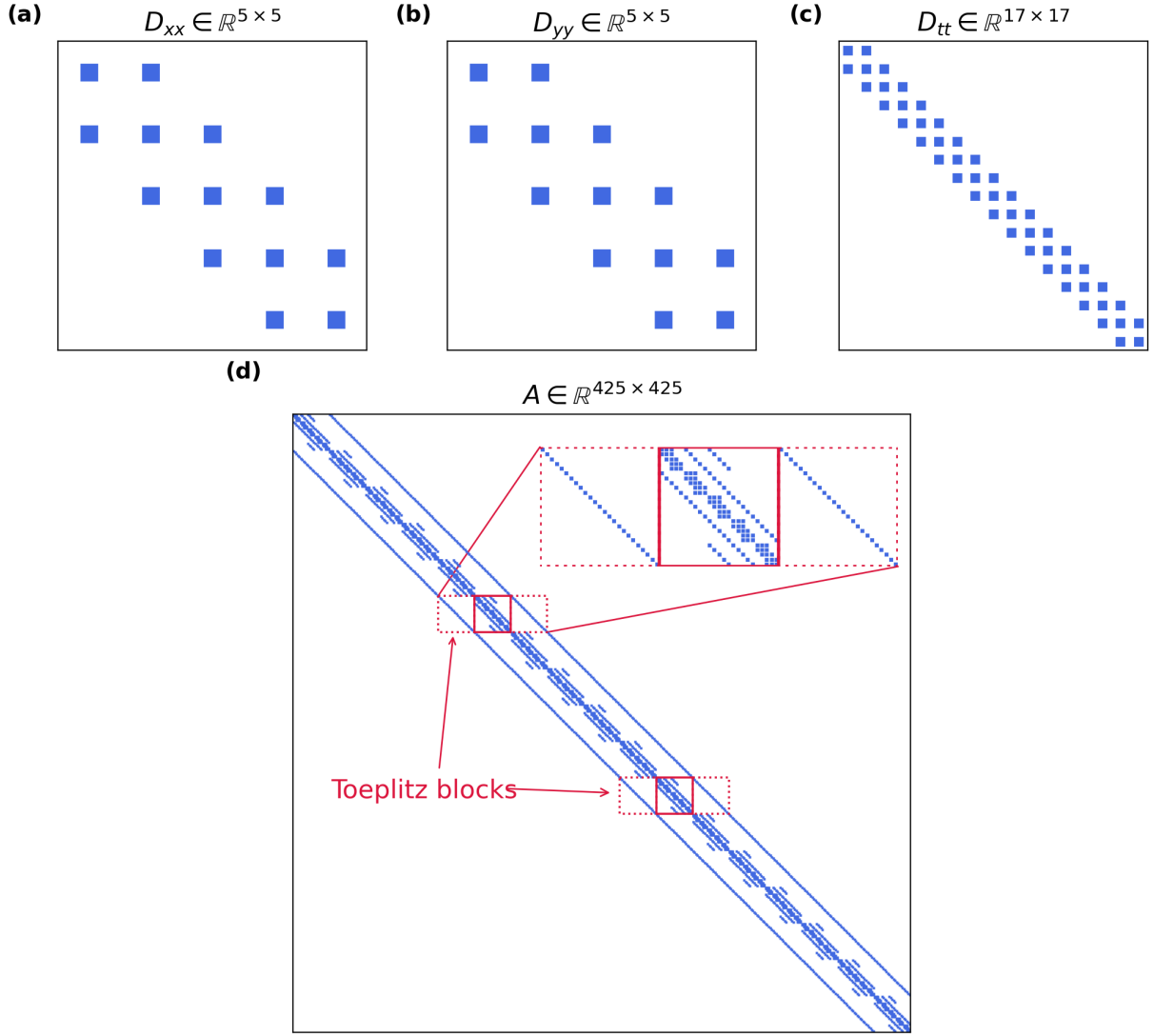


Figure 1: Sparsity plots of the operator matrices D_{xx} , D_{yy} , and D_{tt} (a - c) and the global operator matrix A (d) in the discretised wave equation for a second order centred difference. Blue squares represent the non-zero entries in the matrices, and red boxes indicate the BTTB structure of the global operator. The inset illustrates the block structure: off-diagonal blocks couple neighbouring time levels and the central block represents the current time with the spatial operator terms folded in.

Under this discretisation, the full system can be expressed as a linear system,

$$A\mathbf{u} = \mathbf{s}. \quad (23)$$

2.3.2 Discretisation of the Initial and Boundary Conditions

Initial and boundary conditions are incorporated by modifying the corresponding rows of the discrete operator, such that the complete system represents both the PDE residual and constraint equations within the same global formulation. The initial pressure field is imposed directly by setting the solution at the initial time level to zero, and the zero initial velocity condition is enforced using a finite difference approximation of the first temporal derivative at $t = 0$,

$$u(0, x_i, y_j) = u^0 = 0 \quad (24)$$

$$\frac{\partial}{\partial t} u(0, x_i, y_j) \approx \frac{u^1 - u^0}{\Delta t} = 0 \quad (25)$$

This approximation is rearranged such that it reduces to pinning $u(t_1, x_i, y_j) = 0$.

At the computational boundaries, the interior finite difference stencil is replaced by a discrete approximation of the second-order Higdon absorbing boundary condition (Equation (10)):

$$\delta_{tt}u + (c_1 + c_2)\delta_{tn}u + c_1c_2\delta_{nn}u = 0, \quad (26)$$

where δ_{tt} denotes the discrete second derivative operator in time, δ_{tn} the discrete mixed derivative operator, and δ_{nn} the discrete second derivative of the normal coordinate. These differentiation operators are approximated by first and second order centred finite differences as in Equations (18) - (20).

2.3.3 Discrete Source Injection

Source injection in the discrete domain is handled using Hicks' method of Kaiser-windowed sinc interpolation [18]. This enables sources to be positioned at arbitrary physical coordinates rather than being restricted to the nearest grid node, thereby reducing grid positioning artefacts and improving the accuracy of source injection. Readers are encouraged to consult Hicks' work for more information on the source injection methodology.

2.3.4 Enforcing the Boundary Condition

Centred finite difference stencils of order $2g$ require g nodes on either side of the evaluation points and therefore are not directly applicable within g nodes of the domain boundary. Correspondingly, the domain is extended by g ghost nodes on each side, whose values are populated via Lagrange extrapolation of values on the interior of the domain. The ghost node values are folded into a sparse linear map G that is applied to the space time vector. We apply the boundary conditions as hard constraints by replacing the corresponding rows of the discrete wave operator with the boundary constraint equations. More detail on the construction of this linear map and incorporation of boundary conditions into the linear system is provided in Appendix D.

2.3.5 Solving the ODIL problem

The ODIL problem for the solution of the 2D wave equation is

$$\min_{\mathbf{u}} \|\mathbf{r}(\mathbf{u})\|^2, \quad (27)$$

for the residual $\mathbf{r}(\mathbf{u}) = A\mathbf{u} - \mathbf{s}$. As a baseline, this problem is solved using the low memory bounded BFGS (LBFGSB) method from the SciPy library [35]. The majority of this work discusses an alternative method derived from a Gauss-Newton approach.

In Appendix E, we show that, since the residual is linear in $\mathbf{u}$, the Gauss-Newton update reduces to a single solve of $A\delta u = -r$ for δu , the update to the optimisation field $\mathbf{u}$. For the linear wave equation, applying the Gauss-Newton update is functionally equivalent to solving the discretised linear system directly. The distinction is primarily one of formulation. Instead of solving directly, we pose the problem as a least squares optimisation problem, with the Gauss-Newton method providing the corresponding linear solve.

We use an iterative method to solve for the update $\delta \mathbf{u}$, GMRES. GMRES [30] requires only matrix-vector products, avoiding the requirement for costly factorisation and aligning with our discussed matrix free approach. In addition, GMRES solves $A\delta \mathbf{u} = -r$ directly, which avoids amplifying any ill-conditioning in the system which would be the case for a least squares solve which introduces a transpose term that has the effect of squaring the conditioning number.

The convergence of iterative linear solvers is poor for systems with large conditioning numbers [32]. The GMRES algorithm in particular displays poor convergence properties when the singular values of the system are loosely clustered. The all-at-once wave equation operator is strongly ill-conditioned, with poor clustering of its singular values, as illustrated in Figure 2. Thus we have motivation to precondition our linear system in an attempt to improve the clustering of the singular values such that we may improve the rate of convergence for our linear solve.

2.3.6 Preconditioning the Gauss-Newton Inner Solve

Karnakov et al. [22] suggest that ODIL can be accelerated by multigrid methods. However, such approaches tend to struggle when applied to hyperbolic problems like the wave equation [7, 8], so a different approach is required. We instead employ a circulant preconditioning matrix based on the ParaDiag-II preconditioner [13].

Due to its Toeplitz structure, the temporal operator of Equation (22) differs from a circulant matrix only in its corner entries, and the global BTTB operator differs only by a wraparound block. The ParaDiag-II methodology uses a circulant approximation of the temporal operator to form a preconditioning matrix that exploits the structure of all-at-once wave equation problems.

Circulant matrices are diagonalisable by discrete Fourier transform [17]. This means that the inverse of a circulant matrix can cheaply be computed by calculating its discrete Fourier transform and reciprocating its diagonals. In practice, applying the preconditioner M^{-1} consists of three steps: (i) an α -weighted FFT across the time axis, (ii) N_t independent spatial linear solves, one per mode, and (iii) an inverse α -weighted FFT to recover the solution in the time domain. Here, $\alpha \in (0, 1)$ is a constant, which weights the wrap around blocks in the circulant approximation that will be discussed in further detail shortly. In the Fourier domain, the system is entirely block diagonal, and the coupled system of N_t time steps decouples into N_t independent spatial systems, one per Fourier mode. This makes ParaDiag-II embarrassingly parallel. More information on the diagonalisation of circulant matrices and the α weighting is provided in Appendix F.

The choice of α governs the conditioning of the preconditioned system [13]. The circulant approximation differs from the true temporal operator only in the corner block, weighted by α , so the preconditioned operator $M^{-1}A$ deviates from an identity matrix only by a perturbation whose magnitude is controlled by α . Consequently, the singular values of the preconditioned system cluster tightly around unity rather than spreading across a wide distribution, dramatically improving the conditioning of the system. For all of the results presented in this work, we use $\alpha = 0.001$ to balance numerical stability and solver performance. Figure 2 illustrates the collapse of the singular value distribution after preconditioning.

The LU factorisation of each mode may be precomputed and cached in memory. However, the caching strategy is not free. Since higher source frequencies require finer temporal grids, pre-caching creates a memory demand that grows sharply with frequency (due to large LU factorisations which scale poorly in memory). To support higher frequency sources, we also implement a non-caching approach, where block factorisations are performed "on the fly" at each GMRES iteration. This costs runtime for repeated factorisations, but collapses memory constraints significantly.

We leverage that the discrete Fourier transform of a real valued signal has complex conjugate

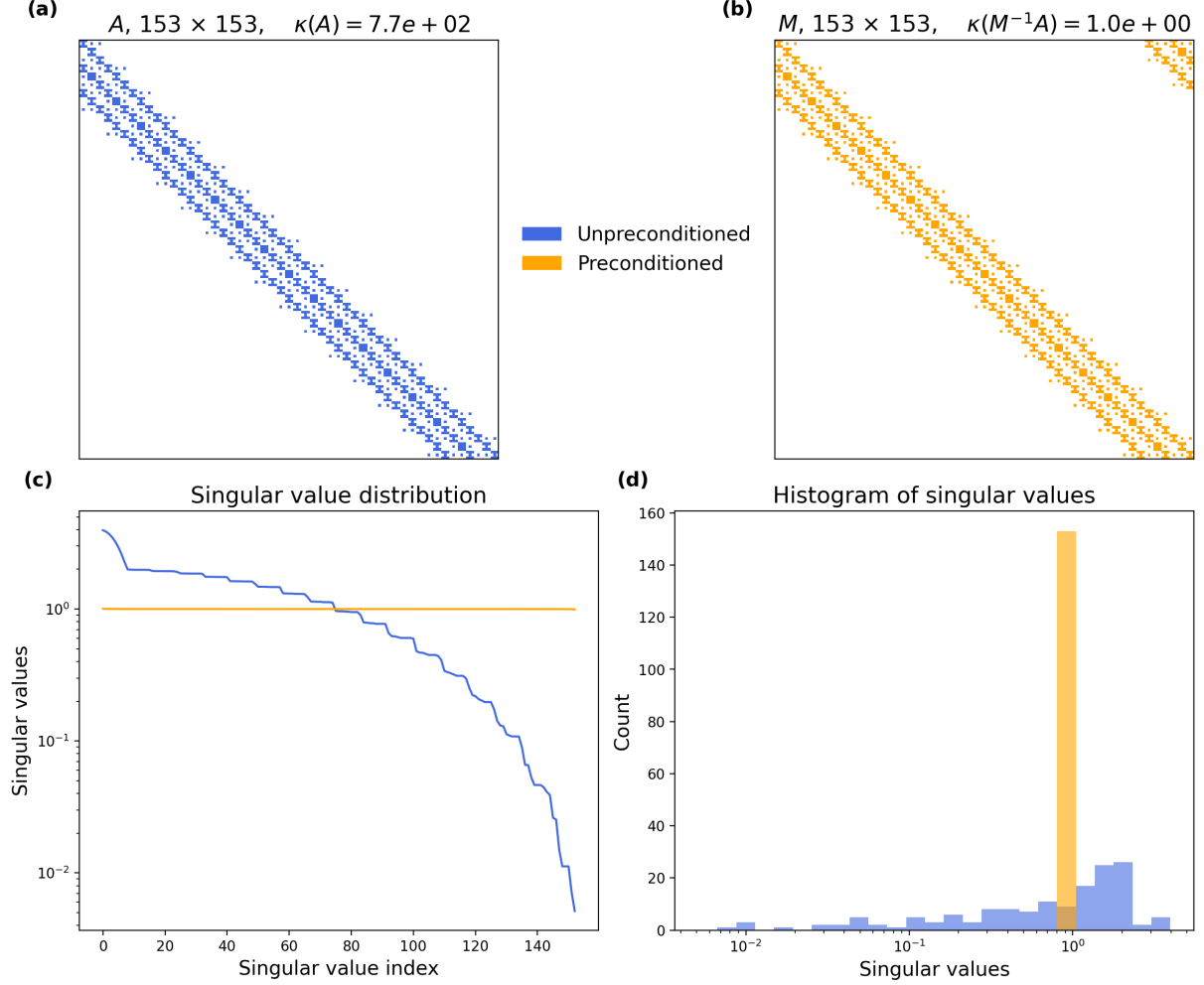


Figure 2: Sparsity plot of the global operator matrix A (a) including boundary equations, and circulant approximation M (b), the preconditioner, along with their singular value κ distributions (c, d).

gate symmetry [21] by only computing factorisations for every other frequency mode, which cuts memory requirements in half. Factorisations are distributed across processes via Python’s `multiprocessing` module. This bypasses the Global Interpreter Lock and allows the $(N_t/2) + 1$ factorisations to run concurrently across CPU cores.

Note that we fix the temporal discretisation at second order for the discussion that follows. Extending to higher time order is a matter of generalising the factorisation and closing the reduced system’s initial conditions with additional starting data, rather than a limitation of the underlying diagonalisation technique, and is left to future work.

2.4 Accuracy Study

To address RQ1 concerning the accuracy properties of ODIL, we benchmark the accuracy of our implementation of the solution of the wave equation using the ODIL framework against an established finite differencing engine, Devito [25]. In order to compare on the basis of accuracy, we must establish a method for obtaining a solution field u_{ref} that is free from bias. Primarily, it must be independent of the grid and scaling choices that both methods enforce (since they differ internally), and, considering both methods use approximate absorbing boundary conditions, free of contamination from any spurious boundary reflections. Note here that our discrete source

injection (Section 2.3.3) is identical to that of Devito.

In a homogeneous domain, an analytical solution to the 2D wave equation with a transient forcing wavelet $s(t)$ can be obtained by convolving the source wavelet in time with the Green's function of the wave equation [36],

$$u(t, \mathbf{x}, \mathbf{x}_s) = s \star g = \int_0^t s(\tau) g(t - \tau, \mathbf{x}, \mathbf{x}_s) d\tau, \quad (28)$$

where $\mathbf{x}_s$ is the location of the source injection, $\mathbf{x}$ is the location of measurement, i.e., the receiver location, and τ is the time of observation.

We use the form of the time-domain Green's function of the two dimensional wave equation described in [10], i.e.,

$$g(t - \tau, r) = \frac{1}{2\pi} \frac{H(t - \tau - r/c)}{\sqrt{(t - \tau)^2 - r^2/c^2}}. \quad (29)$$

where H is the Heaviside function, and $r = \|\mathbf{x} - \mathbf{x}_s\|$ is the spatial distance between a source-receiver pair.

In Appendix G, we show that the convolution integral can be written

$$u(r, t) = \frac{1}{2\pi} \int_0^{\cosh^{-1}(t/a)} s(t - a \cosh(\nu)) d\nu, \quad (30)$$

where $a = r/c$ is the time taken for a wave with speed c to move the source-receiver distance r , and ν is a dimensionless dummy integration variable. Rewriting in this way bypasses the denominator singularity in Equation (29), stabilising the discrete integration. To compute the discrete integral, we use Gauss-Legendre quadrature.

This gives us a means to benchmark the performance of ODIL for solving the wave equation against the Devito solvers by computing metrics on arrival traces observed at discrete locations. We perform a sweep over the number of Degrees of Freedom (DOF) by varying the number of spatial grid nodes. We use $N_x = N_y \in [80, 100, 120, 140, 160, 180, 200]$, with $N_t \in [136, 170, 203, 237, 271, 305]$ computed according to CFL and PPW requirements. For each configuration, we compute the L2 norm of the misfit between the observed and analytical traces at receivers organised in a so called halo configuration (described in Section 2.7). Norms are calculated on traces normalised to unity such they represent the difference in the shapes of the traces independent of differences in wavelet scaling choices.

ODIL and Devito rely on different inexact boundary conditions, so spurious boundary reflection artefacts are present in observed traces. Thus, we restrict trace measurements to a window between the time of arrival for the direct wave, $t_a = r/c$ and an approximation of the arrival time of the first spurious reflection, $t_r = ds + dr/c$, where ds is the distance of the source to the nearest wall, dr is the distance of the receiver to that wall, and c is the constant wave speed in the medium. This prevents differing reflection profiles from contaminating comparisons between the two solution methodologies.

2.5 Performance Study

To address RQ2 concerning computational performance, we measure the wall clock time to solution and memory consumption for ODIL and Devito. We measure memory usage via the

Resident Set Size (RSS), computed using the standard Python library `resource`. We also measure the strong scaling behaviour of ODIL when using the caching and non-caching approaches. IN particular, we measure parallel speedup at n cores, defined as

$$S_n = \frac{T(n=1)}{T(n)}, \quad (31)$$

where T is the wall clock time.

Reported timings represent the wall clock time required to obtain a converged solution, excluding one time initialisation and compilation overheads. For ODIL, this includes the time to compute the sparse factorisations of the block circulant matrices as in Section 2.3.6. For Devito, we run a 'warm-up' solve and exclude this from measurements to exclude JIT compilation overhead from measurements. Each experiment was repeated three times, with the mean runtime reported.

2.6 Study of Wave Equation Specifics

To address RQ3, we look at how ODIL and time stepping treat known sources of error: dispersion and dissipation. In addition, we investigate how the stability of the ODIL solution changes around the CFL limit.

To analyse dispersion, we cross correlate the ODIL and time stepping solutions against the analytical solution at receivers positioned at increasing distance from the source, extracting the lag of peak correlation at each. Since numerical dispersion causes phase error to accumulate with propagation distance, we analyse how these lags change as a function source receiver distance. This relationship isolates the phase velocity error from other contributions to a single lag and serves as our dispersion metric. Dissipation is assessed analogously via the ratio of the numerical and analytical Hilbert envelope amplitudes over the same windowed traces.

To analyse solution stability, we sweep the temporal grid spacing configuration to achieve a certain CFL number C . The CFL number dictates the stability of a numerical simulation of waves, and is given in 2D by

$$C = c\Delta t \sqrt{\frac{1}{\Delta x^2} + \frac{1}{\Delta y^2}}. \quad (32)$$

For a second order explicit time stepping scheme to be stable we require $C \leq 1$ [6].

We assess the stability quantitatively and qualitatively by inspecting solutions on two models at a range of CFL numbers in the range [0.7, 1.3]. On a homogeneous model, we use the analytical solution to analyse how error in the solution behaves about the stability transition zone. To detect instability, we track the maximum field amplitude at each time step and its rate of growth. A solution is classified as unstable if this rate is positive, indicating exponential growth, or if the field becomes non-finite through numerical overflow. We run the same experiment on the Shepp Logan Phantom [31] to qualitatively infer how the quality of the solution degrades past the CFL limit.

2.7 Experimental Configurations

Experiments used a homogeneous model, a circular velocity inclusion, and the Shepp-Logan Phantom [31]. Source and receiver placement varied per experiment. Where referenced, the 'halo' receiver configuration involved an even distribution of receivers on a circle centred on the middle of the domain, and the 'linear' configuration involved an even distribution of receivers along a

vertically centred line. For the sake of reproducibility, all relevant computational parameters are delineated on a per-experiment basis in Appendix H.

All experiments were performed on an AMD Rome compute node from Imperial’s CX3 high performance computing facility. A node is comprised of two AMD EPYC 7742 processors, each with 64 physical cores and 128 threads, and 1Tb of RAM. We control Devito’s parallelism through its Programmatic API for controlling environment variables [1]. All other variables, including compiler optimisations, are left to the default settings. For ODIL, we set the number of concurrent processes to be equal to the number of physical cores prescribed to particular jobs run on CX3.

3 Results

As a baseline, we ran L-BFGS-B, unpreconditioned GMRES, and preconditioned GMRES for a small homogeneous case documented in Table 2. We measured the error norm at a single receiver, along with iteration counts, wall clock execution times, and reasons for exit. The raw results are presented in Appendix I, Table 3. The unpreconditioned GMRES method exited almost immediately because the update δu was smaller than 10^{-12} , a relative tolerance exit criterion. This indicated the optimisation was crawling, and that GMRES without preconditioning is not viable for this problem. While L-BFGS-B achieved a significantly smaller error norm, the number of iterations (and consequently, the wall clock time) required to achieve this reduction in the functional was infeasibly high. In accordance with these results, we exclude L-BFGS-B and the unpreconditioned GMRES methods from further investigation, and consider only the caching and non-caching ODIL methodologies in comparison against Devito.

3.1 Accuracy

In the accuracy study, we perform a sweep over the Degrees Of Freedom (DOF, the number of grid points), and compare the solution achieved by ODIL and Devito against the analytical solution described in Section 2.4. In Appendix I, Table 4 we report the mean of the L2 norm of the normalised trace misfit for each of the 24 receivers in the homogeneous domain for both methodologies. In Figure 3, we illustrate the accuracy convergence against DOF, along with the per-receiver error, and some sample traces which also indicate the spurious reflections that are excluded from calculations by the pre-reflection window.

Error norm measurements demonstrate that the ODIL methodology converges to the same solution as the time stepping approach. This is expected, since both methods rely on the same discretisation scheme. The accuracy of both methods refines with the number of degrees of freedom due to an increasing number of points per wavelength, as illustrated by Figure 3a. The error that remains represents numerical modelling errors resulting from the discretisation, along with diffusion and dissipation error. The results validate that the preconditioned ODIL methodology is capable of solving the forced acoustic wave equation in two dimensions to the same degree of accuracy as a time stepping methodology. This closes RQ1.

3.2 Computational Performance Properties

We begin our evaluation of performance with a comparison of algorithms. This analysis is used to define bounds on the time and space complexity of each algorithm, which are used as reference points for understanding the runtime and memory performance of the ODIL and time stepping methodologies. Following this, we investigate the strong scaling behaviour of the methods.

The operations that dominate the runtime and memory cost of the ODIL routine are the repeated sparse LU factorisations executed in the preconditioner. The factorisation is orchestrated by

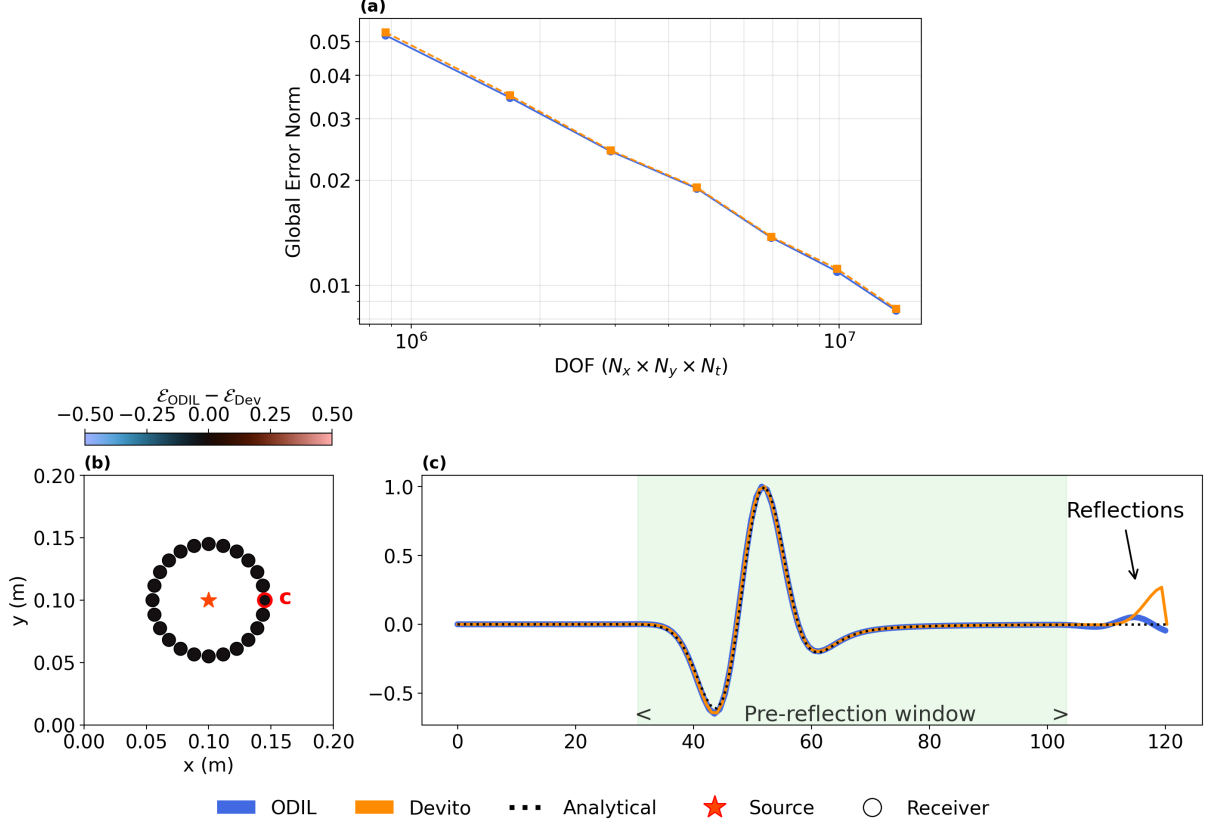


Figure 3: Panel (a) indicates global error norm as a function of Degrees of Freedom (DOF). Panel (b) illustrates the relative difference between the normalised trace residual norms of ODIL and Devito. Panel (c) shows waveforms observed at the receiver indicated by the red circle in Panel (b) against the analytical solution. The trace panel also demonstrates spurious boundary reflections and the pre-reflection window which omits them from metric calculations. Panels (b) and (c) visualise the results on the coarsest grid (lowest DOF) of $N_z = N_y = 80$, giving approximately 10 points per wavelength.

SuperLU [9] which uses Gaussian elimination with partial pivoting [24]. Exact space and time complexities for SuperLU are difficult to pin down, so instead we compare against bounds from the literature. A conservative worst case bound on the complexity is that of naive Gaussian elimination on an $n \times n$ dense matrix: $O(n^3)$ in time and $O(n^2)$ in space. For SuperLU on a sparse matrix with partial pivoting and effective row ordering, a suitable upper bound was proposed by Alan G. and Esmond N. [14]: $O(n^{3/2})$ in time and $O(n \log n)$ in space.

For explicit time stepping, determining complexity is straightforward. In a time stepping routine, for each grid point we require a fixed number of FLOPS due to a fixed stencil width. For an $N_x \times N_y$ 2D spatial domain with $N_s = N_x \times N_y$ spatial nodes simulated over N_t time steps, the time complexity is therefore $O(N_s N_t)$. Time stepping approaches need not store the entire $N_s \times N_t$ wavefield, but can instead use rolling buffers (as Devito indeed does [25]), so the space complexity is merely $O(N_s)$.

Figure 4 presents the serial performance of the ODIL methodologies against Devito. Figures 4a and c demonstrate that our implementation of ODIL sits within the time and space complexity bounds discussed in this work, while Devito’s initial departure from the proposed complexities for time stepping is likely due to Python process overheads dominating the measurements. Figure 4b demonstrates raw wall clock measurements and evidences that, when run in serial, ODIL severely underperforms Devito, which is primarily attributable to inferior algorithmic scalability.

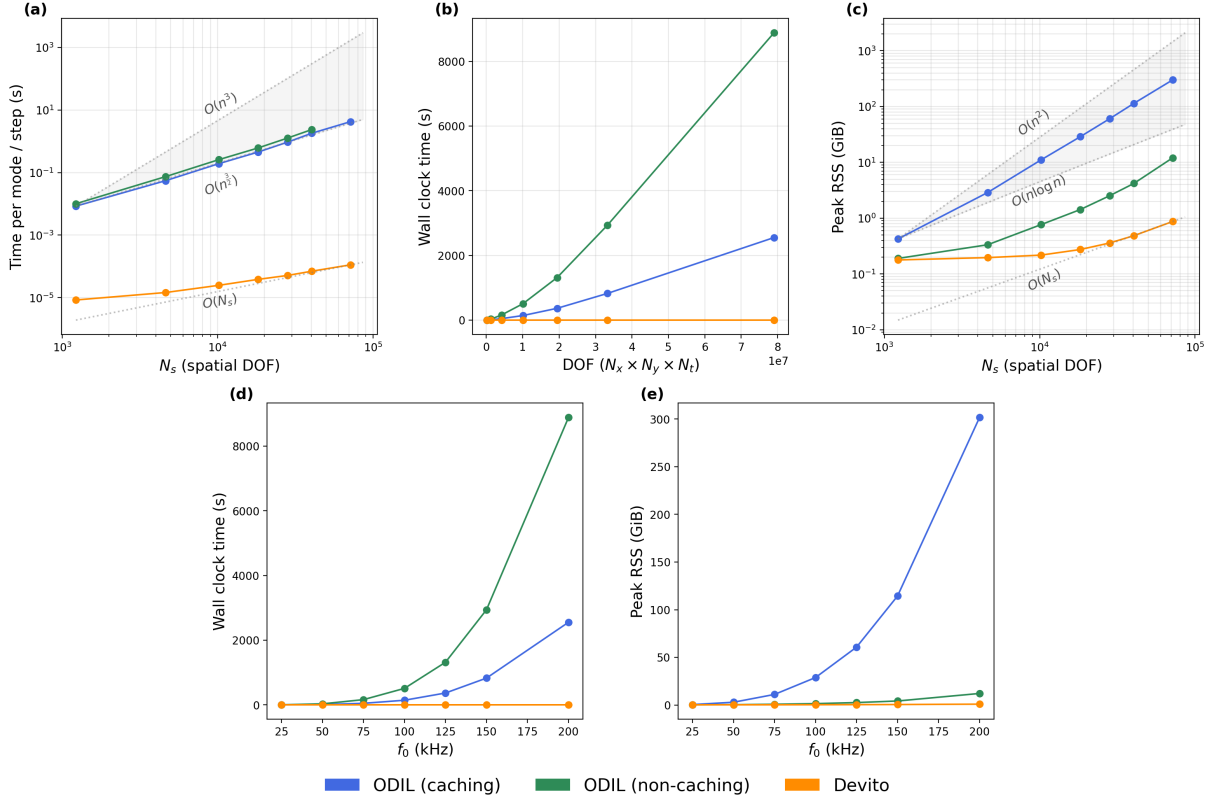


Figure 4: Serial (single core) runtime and memory performance behaviour, with comparisons to the discussed complexity bounds (signified by dashed lines/ grey regions).

In Figure 4b, the non-caching method sits below the lower bounds of memory complexity, and this is expected. Since factors are discarded after they are formed, the peak memory usage cannot exceed $M_{\max} = p \times M_k$, where p is the number of processes and M_k is the memory required to factorising a Toeplitz block corresponding to a single frequency mode. Figure 4c, which isolates the per mode factorisation wall clock cost, shows that run time performance sits very close to the proposed lower bound on time complexity. This indicates near-optimal row ordering in the factorisation stage.

Figures 4c and d illustrate wall clock time and RSS as a function of peak source frequency f_0 . It demonstrates a key limitation of the preconditioner we have applied. The temporal grid density depends on the peak frequency it must resolve according to the Nyquist criterion, and, because the ParaDiag II preconditioner scales poorly in memory when factorisations are cached, it limits the frequencies which can be used in practice. If you require frequencies in the ultrasound range and cannot access a large amount of RAM, you must use the uncached methodology. In that case, the factorisation stage is likely to take an impractical amount of time to complete.

Figure 5a demonstrates the strong scaling behaviour of the three methodologies. ODIL demonstrates near-linear speedup up to 32 cores. The AMD Rome node used to execute the code has 4 NUMA domains, each formed of 32 cores. Thus it is reasonable to assume the departure from linear speedup is due to inefficiencies introduced by NUMA domain crosstalk. Speedup for Devito is negligible, which we attribute to MPI overhead undoing any of the (relatively small) potential speedup gains for this problem. Figure 5c demonstrates that, for this problem, exploiting the parallelism in preconditioner allows ODIL to compete more closely with time stepping, though the lowest wall clock time measurement (at 128 cores/processes) is still ~ 10 times greater than that of Devito.

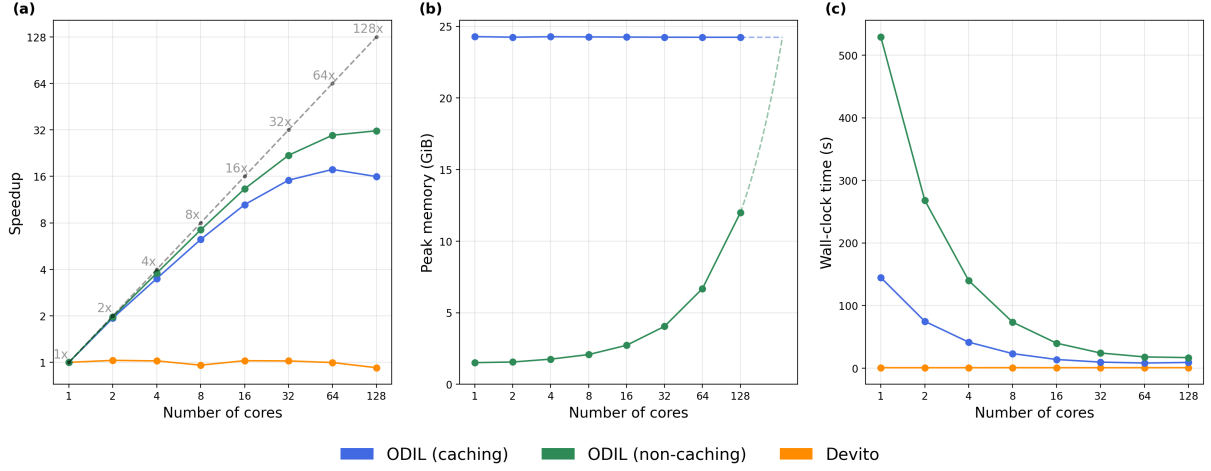


Figure 5: Strong scaling behaviour of (caching and non-caching) ODIL versus Devito: parallel speedup (a) and memory consumption as a function of the number of cores (b). Panel (c) illustrates raw wall clock times as a function of the number of processes. The dashed lines in panel (b) are extrapolated values demonstrating the convergence in memory consumption of the non-caching method to that of the caching method.

Figure 5b demonstrates a less obvious idiosyncrasy of the non-caching methodology. As mentioned, $M_{\max}$ depends on the number of processes, while the caching methodology has a constant cost directly proportional to the number of modes. As a result, when the number of processes is equal to the number of modes to be factorised, the memory consumption of the non-caching method converges to that of the caching method.

3.3 Wave Equation Specifics

4 Discussion

5 Conclusion

Appendix

A Expansion of the Higdon Boundary Condition

Starting from the Higdon boundary condition (Equation (7)), we define the interior operator

$$v := \left(\frac{\partial}{\partial t} + c_2 \frac{\partial}{\partial n} \right) u, \quad (33)$$

using which the Higdon condition can be written

$$\left(\frac{\partial}{\partial t} + c_1 \frac{\partial}{\partial n} \right) v = 0. \quad (34)$$

Applying the outer operator,

$$\frac{\partial v}{\partial t} + c_1 \frac{\partial v}{\partial n} = 0, \quad (35)$$

and substituting v we arrive at

$$\frac{\partial}{\partial t} \left(\frac{\partial u}{\partial t} + c_2 \frac{\partial u}{\partial n} \right) + c_1 \frac{\partial}{\partial n} \left(\frac{\partial u}{\partial t} + c_2 \frac{\partial u}{\partial n} \right) = 0. \quad (36)$$

Differentiating each term gives

$$\frac{\partial^2 u}{\partial t^2} + c_2 \frac{\partial^2 u}{\partial t \partial n} + c_1 \frac{\partial^2 u}{\partial n \partial t} + c_1 c_2 \frac{\partial^2 u}{\partial n^2} = 0 \quad (37)$$

and since mixed derivatives commute we can write

$$\frac{\partial^2 u}{\partial t \partial n} = \frac{\partial^2 u}{\partial n \partial t}, \quad (38)$$

and rearrange to find

$$\frac{\partial^2 u}{\partial t^2} + (c_1 + c_2) \frac{\partial^2 u}{\partial t \partial n} + c_1 c_2 \frac{\partial^2 u}{\partial n^2} = 0, \quad (39)$$

which is the result presented in Equation (10).

B Central Difference Schemes

On the discrete computational domain described in Section 2.3, the temporal and spatial derivatives in the wave equation (Equation 1) are approximated using standard central difference schemes of orders 2, 4, 6, and 8. For a $2P + 1$ point stencil in x , the general form of the centred difference is

$$\frac{\partial^2}{\partial x^2} u(t_n, x_i, y_j) \approx \frac{1}{\Delta x^2} \sum_{p=-P}^P c_p u_{i+m}, \quad (40)$$

which reads analogously in y and t . The coefficients c_p are obtained via the usual Taylor series construction and are chosen to achieve second, fourth, and eighth order accuracy. Those coefficients are tabulated below.

Order	Offsets	Coefficients
2nd	-1, 0, 1	1, -2, 1
4th	-2, -1, 0, 1, 2	$-\frac{1}{12}, \frac{4}{3}, -\frac{5}{2}, \frac{4}{3}, -\frac{1}{12}$
6th	-3, -2, -1, 0, 1, 2	$\frac{1}{90}, -\frac{3}{20}, \frac{3}{2}, -\frac{49}{18}, \frac{3}{2}, -\frac{3}{20}, \frac{1}{90}$
8th	-4, -3, -2, -1, 0, 1, 2, 3, 4	$-\frac{1}{560}, \frac{8}{315}, -\frac{1}{5}, \frac{8}{5}, -\frac{205}{72}, \frac{8}{5}, -\frac{1}{5}, \frac{8}{315}, -\frac{1}{560}$

Table 1: Coefficients for centred finite difference approximations of the second derivative.

C Formation of the Laplacian Matrix Operator

The discrete Laplacian operator is constructed using Kronecker products according to

$$L_{xy} := D_{xx} \otimes I_y + I_x \otimes D_{yy}, \quad (41)$$

where $L_{xy} \in \mathbb{R}^{N_x N_y \times N_x N_y}$ acts on the vectorised spatial wave field. Here, $I_x \in \mathbb{R}^{N_x \times N_x}$ and $I_y \in \mathbb{R}^{N_y \times N_y}$ are the identity matrices.

D Matrix Form of the Boundary Operator

The described ghost node reconstruction is a linear operation. It can therefore be represented as a sparse matrix

$$\mathbf{u}_{\text{full}} = G\mathbf{u}, \quad (42)$$

where $G \in \mathbb{R}^{(n+2g) \times n}$ maps the n interior unknowns to the extended vector containing both interior and ghost nodes. The matrix G consists of an identity block corresponding to the interior nodes and additional rows containing the Lagrange extrapolation weights for the ghost nodes.

Accordingly, in the x direction, we can write

$$D_x^{(g)} = D_x G, \quad D_{xx}^{(g)} = D_{xx} G, \quad (43)$$

which yields ordinary square difference operators, valid at every node including the boundary, with the extrapolated ghost contribution folded in. We broadcast the discrete operators δ_{tt} , δ_{tn} , δ_{nn} of Equation (26) to space-time by means of another Kronecker product as in Equation (2), for instance

$$D_{tt} = \delta_{tt} \otimes I_{xy}, \quad (44)$$

and correspondingly for the spatial operators. Then, restricting to the n_b nodes on edge b with a selection matrix R_b , we can compute the discrete normal derivative operators as

$$D_n = R_b D_x^{(g)}, \quad D_{nn} = R_b D_{xx}^{(g)}, \quad (45)$$

with $D_x^{(g)}$ replaced by $D_y^{(g)}$ on the horizontal edges, and where $\cdot^{(g)}$ denotes that the operator acts on boundary nodes. Since ∂_n is the outward normal, D_n is signed according to which edge is being treated. Outward is taken to be negative at the left/bottom boundary and positive at the right/top, so D_n flips sign accordingly.

Equation (26) can then be written using the matrix operators,

$$D_{tt}\mathbf{u}_b + (c_1 + c_2)D_{tn}\mathbf{u}_b + c_1 c_2 D_{nn}\mathbf{u}_b = 0, \quad (46)$$

where $\mathbf{u}_b$ are the boundary columns of the solution array.

E Reduction of the Gauss Newton Method

Gauss-Newton is an iterative method for solving nonlinear least squares optimisation problems that approximates the Hessian of the objective function, avoiding the computation of second derivatives. At the k -th iterate, the residual is linearised as

$$\mathbf{r}(\mathbf{u}_k + \delta\mathbf{u}) \approx \mathbf{r}_k + J_k \delta\mathbf{u}, \quad (47)$$

where J_k is the Jacobian of the residual evaluated at $\mathbf{u}_k$. Minimising the resulting linearised least squares problem gives the Gauss-Newton normal equations,

$$J_k^T J_k \delta \mathbf{u}_k = -J_k^T \mathbf{r}_k. \quad (48)$$

For the present problem, however, the residual is linear in $\mathbf{u}$, such that $J_k = A$, where A is the constant global discretisation matrix of our all-at-once system. The Gauss-Newton equations therefore reduce to

$$A^T A \delta \mathbf{u}_k = -A^T \mathbf{r}_k. \quad (49)$$

Assuming that the discretised wave equation is uniquely solvable, such that A is non-singular, A^T is then also non-singular. Resultingly, Equation (48) can be written

$$A^T (A \delta \mathbf{u} + r) = 0. \quad (50)$$

since A^T is necessarily non-zero, the minimiser of the Gauss-Newton equations can be found by solving

$$A \delta \mathbf{u} = -r, \quad (51)$$

for the update, $\delta \mathbf{u}$. Using the definition of the residual $\mathbf{r}$, this can be written as

$$A(\mathbf{u}_k + \delta \mathbf{u}) = \mathbf{s}, \quad (52)$$

so a single update $\delta \mathbf{u}$ to the current iterate $\mathbf{u}_k$ gives the exact solution to the linear wave equation. This is a reflection of the property of the Gauss-Newton algorithm that it converges in exactly one outer iteration for linear problems [12].

F Diagonalisation of the α -circulant preconditioner

A circulant matrix $\tilde{D}_{tt}$ can be diagonalised by the eigendecomposition

$$\tilde{D}_{tt} = F^{-1} \Lambda F, \quad (53)$$

where F is the discrete Fourier transform matrix and Λ is a diagonal matrix containing the eigenvalues of $\tilde{D}_{tt}$. The ParaDiag II method uses an α circulant matrix whose wraparound block is scaled by some constant $\alpha \in (0, 1)$. As a result, the appropriate diagonalisation uses a weighted Fourier transform. With the diagonal scaling matrix $D_\alpha = \text{diag}(\alpha^{l/N_t})$ for $l = 0, \dots, N_t - 1$, we have

$$\tilde{D}_{tt} = D_\alpha^{-1} F^{-1} \Lambda_\alpha F D_\alpha, \quad (54)$$

where Λ_α is now diagonal with entries depending on α .

G Convolution Integral Reduction

In Equation (30) we introduce a reduced form of the convolution integral

$$u(t, \mathbf{x}, \mathbf{x}_s) = s \star g = \int_0^t s(\tau) g(t - \tau, \mathbf{x}, \mathbf{x}_s) d\tau, \quad (55)$$

that evaluates the convolution of the Ricker source term (Equation (3)) with the analytical Green's function for the 2D wave equation (Equation (29)),

$$g(t - \tau, r) = \frac{1}{2\pi} \frac{H(t - \tau - r/c)}{\sqrt{(t - \tau)^2 - r^2/c^2}}. \quad (56)$$

The denominator of the Green's function can become singular, leading to instability in the discrete integration. Thus we have motivation to find an equivalent form of the convolution integral that avoids this singularity. In the following derivation, we show how the Green's function can be rewritten by means of a hyperbolic substitution.

With $a = r/c$, the time taken for a wave to move a distance r through a homogeneous medium with wave speed c , the Heaviside function reads

$$H(t - \tau - a) = \begin{cases} 1 & t - \tau - a > 0 \\ 0 & \text{otherwise} \end{cases} \quad (57)$$

and so we require

$$t - \tau - a > 0 \iff \tau < t - a. \quad (58)$$

Thus, our upper integration limit reduces to $t - a$ up to which $H = 1$, leading to the explicit integral

$$u(r, t) = \frac{1}{2\pi} \int_0^{t-a} \frac{s(\tau)}{\sqrt{(t - \tau)^2 - a^2}} d\tau. \quad (59)$$

If we define the duration $\mathcal{T} = t - \tau$, then the differential and limits change:

$$\begin{aligned} \frac{d\mathcal{T}}{d\tau} = -1 &\implies d\mathcal{T} = -d\tau \\ \tau \in [0, t - a] &\implies \mathcal{T} \in [t, a] \end{aligned}$$

Making the substitution $\tau = t - \mathcal{T}$ and $d\tau = -d\mathcal{T}$, we have

$$u(r, t) = \frac{1}{2\pi} \int_t^a \frac{s(t - \mathcal{T})}{\sqrt{\mathcal{T}^2 - a^2}} - d\mathcal{T}, \quad (60)$$

To make our interval increasing, we negate the interval and flip the limits, cancelling the negative the differential introduced, yielding

$$u(r, t) = \frac{1}{2\pi} \int_a^t \frac{s(t - \mathcal{T})}{\sqrt{\mathcal{T}^2 - a^2}} d\mathcal{T}, \quad t > a. \quad (61)$$

The intuitive explanation of the limits here is that we are convolving the Ricker wavelet and the Green's function between the arrival time $a = r/c$ and the current time t .

Now we are finally in the position to make a cosh substitution to remove the singular denominator. Let $\mathcal{T} = a \cosh \nu$ for the dimensionless dummy integration variable $\nu : \nu \geq 0$. Then

$$\frac{d\mathcal{T}}{d\nu} = a \sinh(\nu) \quad \Rightarrow \quad d\mathcal{T} = a \sinh(\nu) d\nu, \quad (62)$$

$$\mathcal{T} \in [a, t] \quad \Rightarrow \quad \nu \in \left[\cosh^{-1} \left(\frac{a}{a} \right), \cosh^{-1} \left(\frac{t}{a} \right) \right] \quad (63)$$

$$= \left[0, \cosh^{-1} \left(\frac{t}{a} \right) \right] \quad (64)$$

and the denominator in the integral becomes

$$\sqrt{\mathcal{T}^2 - a^2} = \sqrt{a^2 \cosh^2(\nu) - a^2} = a \sqrt{\cosh^2(\nu) - 1} = a \sinh \nu, \quad (65)$$

by the identity $\cosh^2(x) - \sinh^2(x) \equiv 1$. Substituting in these results yields

$$u(r, t) = \frac{1}{2\pi} \int_0^{\cosh^{-1}(t/a)} \frac{s(t - a \cosh(\nu))}{a \sinh(\nu)} (a \sinh(\nu) d\nu), \quad (66)$$

which simplifies nicely down to the result stated in Equation (30):

$$u(r, t) = \frac{1}{2\pi} \int_0^{\cosh^{-1}(t/a)} s(t - a \cosh(\nu)) d\nu. \quad (67)$$

H Experimental Configurations

This section summarises the numerical and physical parameters used across each of the described experiments. Key parameters are summarised in Tables 2a and 2b, respectively, while a short discussion of other parameters follows below.

Where referenced, the homogeneous model had $c = 1500 \text{ m s}^{-1}$. The so called 'inclusion' model featured a circular velocity anomaly of $c = 1800 \text{ m s}^{-1}$ against a background of $c = 1500 \text{ m s}^{-1}$, centred on the middle of the relevant domain with a radius of 0.03 m. The Shepp Logan Phantom model was derived from the scikit-image [34] API and was manipulated such that the skull had a wave speed of 2800 m s^{-1} , the brain tissue wave speed was in the range $[1500, 1600] \text{ m s}^{-1}$, and the background wave speed was 1480 m s^{-1} .

Where referenced in Table 2b, the 'halo' receiver configuration is comprised of an array of n receivers distributed evenly on circle with radius r centred on the middle of the domain. The 'linear' configuration is comprised a uniform distribution of n receivers along a vertically centred line in the x direction.

Other parameters include: $\alpha = 10^{-3}$ for the preconditioner, absolute tolerance and relative tolerance of 10^{-10} and 10^{-12} for the GMRES iterative solve, respectively, and a minimum number of Points Per Wavelength (PPW) of 10. For the stability experiments, we used finite difference stencils with time and space order of 2. For the remaining experiments, a space order of 6 was used, with a time order of 2. All data were stored in double precision.

Experiment	Domain (m)	Duration (μ s)	N_x	N_y	N_t	DOF
Baseline	$[0, 0.1]^2$	113	68	68	231	1.07×10^6
Accuracy	$[0, 0.2]^2$	120	varied	varied	varied	varied
Serial performance	$[0, 0.2]^2$	226	varied	varied	varied	varied
Strong scaling	$[0, 0.2]^2$	226	100	100	553	5.53×10^6
Dispersion	$[0, 0.2]^2$	226	varied	varied	varied	varied
Stability (quantitative)	$[0, 0.1]^2$	113	68	68	varied	varied
Stability (qualitative)	$[0, 0.3]^2$	339	241	241	varied	varied

(a) Numerical discretisation parameters.

Experiment	Model	Source loc. (m)	Receiver config.	f_0 (kHz)
Baseline	Homogeneous	(0.05, 0.05)	$n = 1$, (0.08, 0.05) m	100
Accuracy	Homogeneous	(0.10, 0.10)	$n = 24$, halo*, $r = 0.045$ m	50
Serial performance	Inclusion*	(0.10, 0.10)	NA	varied
Strong scaling	Inclusion*	(0.10, 0.10)	NA	50
Dispersion	Homogeneous	(0.05, 0.05)	$n = 9$, linear* $x = 0.05 + 0.01i$ m, $i = 1, \dots, 9$	100
Stability (quantitative)	Homogeneous	(0.05, 0.05)	$n = 1$, (0.08, 0.05) m	100
Stability (qualitative)	Shepp Logan Phantom*	(0.09, 0.24)	$n = 1$, (0.22, 0.08) m	100

(b) Physical and acquisition parameters.

Table 2: Computational parameters used in the accuracy experiments. An NA entry signifies that a parameter is Not Applicable for the experiment it pertains to. * for more notes on model and receiver configurations, consult the discussion below.

I Results Tables

Optimiser	$\bar{\mathcal{E}}$	Wall clock (s)	Iterations	Exit reason
LBFGSB	0.26854	222.6	1000	Iteration ceiling hit
GMRES (unpreconditioned)	1.41414	11.1	4	Tiny updates (crawling solve)
GMRES (preconditioned)	0.08261	4.8	1	Converged

Table 3: Results of the baseline measurements discussed in Section 3. $\mathcal{E}$ denotes the global error norm.

N_x, N_y	N_t	DOF	Δx	PPW	$\bar{\mathcal{E}}_{\text{ODIL}}$	$\bar{\mathcal{E}}_{\text{Devito}}$	GMRES iters.
80	136	870,400	0.002532	11.85	0.052191	0.053204	3
100	170	1,700,000	0.002020	14.85	0.034569	0.035088	3
120	203	2,923,200	0.001681	17.85	0.024245	0.024403	3
140	237	4,645,200	0.001439	20.85	0.018974	0.019090	3
160	271	6,937,600	0.001258	23.85	0.013713	0.013797	3
180	305	9,882,000	0.001117	26.85	0.010958	0.011150	3
200	339	13,560,000	0.001005	29.85	0.008472	0.008572	3

Table 4: Results of the accuracy sweep presented in Figure 3. $\bar{\mathcal{E}}_{\text{ODIL}}$ and $\bar{\mathcal{E}}_{\text{Devito}}$ denote the mean of the global error norm averaged over 3 repeated measurements. The GMRES iterations for the internal $A\delta u = -r$ solve in the Gauss-Newton methodology that ODIL employs are also reported.

References

- [1] Configuration System | xdslproject/devito. <https://deepwiki.com/xdslproject/devito/2.2-configuration-system>.
- [2] Well-posedness and Stability. In Bertil Gustafsson, editor, *High Order Difference Methods for Time Dependent PDE*, pages 13–68. Springer, Berlin, Heidelberg, 2008.
- [3] Yajun An. Uniform dispersion reduction schemes for the one dimensional wave equation in isotropic media. *Journal of Computational Physics*, 341:13–21, July 2017.
- [4] Christine Bernardi and Yvon Maday. Spectral methods. In *Techniques of Scientific Computing (Part 2)*, volume 5 of *Handbook of Numerical Analysis*, pages 209–485. Elsevier, January 1997.
- [5] Gary C. Cohen. The Basic Equations. In Gary C. Cohen, editor, *Higher-Order Numerical Methods for Transient Wave Equations*, pages 3–13. Springer, Berlin, Heidelberg, 2002.
- [6] R. Courant, K. Friedrichs, and H. Lewy. On the Partial Difference Equations of Mathematical Physics. *IBM Journal of Research and Development*, 11(2):215–234, March 1967.
- [7] Hussam Al Daas, Niall Bootland, Tyrone Rees, and Sue Thorne. A Review of Time Stepping Techniques and Preconditioning for Hyperbolic and Anisotropic Elliptic Problems.
- [8] Federico Danieli and Andrew J. Wathen. All-at-once solution of linear wave equations.
- [9] James W. Demmel, John Gilbert, and Xiaoye S. Li. SuperLU Users’ Guide. Technical Report UCB/CSD-97-944, May 1997.
- [10] Dean G. Duffy. *Green’s Functions with Applications*. Chapman and Hall/CRC, New York, 2 edition, March 2015.
- [11] Bezalel Finkelstein and Raphael Kastner. Finite difference time domain dispersion reduction schemes. *Journal of Computational Physics*, 221(1):422–438, January 2007.
- [12] R. Fletcher. Newton-Like Methods. In *Practical Methods of Optimization*, chapter 3, pages 44–79. John Wiley & Sons, Ltd, 2000.
- [13] Martin J. Gander, Jun Liu, Shu-Lin Wu, Xiaoqiang Yue, and Tao Zhou. ParaDiag: Parallel-in-time algorithms based on the diagonalization technique. <https://arxiv.org/abs/2005.09158v4>, May 2020.

- [14] Alan George and Esmond Ng. On the complexity of sparse QR and LU factorization of finite-element matrices. *SIAM Journal on Scientific and Statistical Computing*, 9(5):849–861, 1988.
- [15] A. Gillman and P. G. Martinsson. A Direct Solver with $O(N)$ Complexity for Variable Coefficient Elliptic PDEs Discretized via a High-Order Composite Spectral Collocation Method. *SIAM Journal on Scientific Computing*, 36(4):A2023–A2046, January 2014.
- [16] S. Hao. Numerical Methods for Solving Linear Elliptic PDEs: Direct Solvers and High Order Accurate Discretizations. 2015.
- [17] M. Hariprasad and Murugesan Venkatapathi. Circulant decomposition of a matrix and the eigenvalues of Toeplitz type matrices. *Applied Mathematics and Computation*, 468:128473, May 2024.
- [18] Graham. J. Hicks. Arbitrary source and receiver positioning in finite-difference schemes using Kaiser windowed sinc functions. *Geophysics*, 67(1):156–166, January 2002.
- [19] Robert L. Higdon. Numerical absorbing boundary conditions for the wave equation. *Mathematics of Computation*, 49(179):65–90, 1987.
- [20] Ilse C. F. Ipsen and Carl D. Meyer. The Idea Behind Krylov Methods. *The American Mathematical Monthly*, 105(10):889–899, December 1998.
- [21] Steven G. Johnson and Matteo Frigo. A Modified Split-Radix FFT With Fewer Arithmetic Operations. *IEEE Transactions on Signal Processing*, 55(1):111–119, January 2007.
- [22] Petr Karnakov, Sergey Litvinov, and Petros Koumoutsakos. Solving inverse problems in physics by optimizing a discrete loss: Fast and accurate learning without neural networks. *PNAS Nexus*, 3(1):pgae005, January 2024.
- [23] Robert C. Kirby and Lawrence Mitchell. Solver Composition Across the PDE/Linear Algebra Barrier. *SIAM Journal on Scientific Computing*, 40(1):C76–C98, January 2018.
- [24] Xiaoye S. Li. An overview of SuperLU: Algorithms, implementation, and user interface. *ACM Transactions on Mathematical Software (TOMS)*, 31(3):302–325, September 2005.
- [25] Mathias Louboutin, Michael Lange, Fabio Luporini, Navjot Kukreja, Philipp A. Witte, Felix J. Herrmann, Paulius Velesko, and Gerard J. Gorman. Devito (v3.1.0): An embedded domain-specific language for finite differences and geophysical exploration. *Geoscientific Model Development*, 12(3):1165–1187, March 2019.
- [26] Robert A. Marshall and Umran S. Inan. Numerical dispersion and dissipation. In *Numerical Electromagnetics: The FDTD Method*, pages 132–151. Cambridge University Press, Cambridge, 2011.
- [27] Prasad Pranjal and David J. Silvester. Optimal iterative solvers for linear nonsymmetric systems and nonlinear systems with PDE origins: Balanced black-box stopping tests. <https://eprints.maths.manchester.ac.uk/2639/>, May 2018.
- [28] Norman Ricker. Wavelet functions and their polynomials. *Geophysics*, 9(3):314–323, July 1944.
- [29] Milton E. Rose. Finite Difference Schemes for Differential Equations. *Mathematics of Computation*, 18(86):179–195, 1964.
- [30] Youcef Saad and Martin H. Schultz. GMRES: A generalized minimal residual algorithm for solving nonsymmetric linear systems. *SIAM Journal on Scientific and Statistical Computing*, 7(3):856–869, 1986.

- [31] L. A. Shepp and B. F. Logan. The Fourier reconstruction of a head section. *IEEE Transactions on Nuclear Science*, 21(3):21–43, June 1974.
- [32] Lloyd N. Trefethen and David Bau, III. Chapter III: Conditioning and Stability. In *Numerical Linear Algebra*, Other Titles in Applied Mathematics, pages 89–144. Society for Industrial and Applied Mathematics, January 1997.
- [33] M. J. TURNER, R. W. CLOUGH, H. C. MARTIN, and L. J. TOPP. Stiffness and Deflection Analysis of Complex Structures. *Journal of the Aeronautical Sciences*, 23(9):805–823, 1956.
- [34] Stéfan van der Walt, Johannes L. Schönberger, Juan Nunez-Iglesias, François Boulogne, Joshua D. Warner, Neil Yager, Emmanuelle Gouillart, and Tony Yu. Scikit-image: Image processing in Python. *PeerJ*, 2:e453, June 2014.
- [35] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C. J. Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, and Paul van Mulbregt. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, March 2020.
- [36] Kees Wapenaar. Multiple Reflections on Huygens’ Principle. *Geophysical Prospecting*, 73(6):e70038, 2025.